

Quiz: técnica de paralelización

Juan Sebastian Manrique Moreno – 20202107016

Archivos: serie-000.c y parallel-000.c

Las instrucciones son las siguientes:

1. Calcular C como la suma de los vectores A y B, y luego sumar todos los elementos de C
2. Distribuir las iteraciones del bucle en bloques de 3 elementos de manera estática
3. Imprimir en pantalla el número de hilo con cada resultado

El código realizado es:

```
/* parallel-000.c - Juan Sebastian Manrique Moreno
 * Vector C como suma de vectores A y B.
 * Variable total acumula valores usando reducción
 * Uso de directiva combinada para paralelizar bucle
 */

#include <stdio.h>
#include <omp.h>

int main(){
    /* Se obtiene el número de hilo maestro y se imprime */
    int m = omp_get_thread_num();
    printf("master thread=%d in serial region\n", m);

    /* Se declaran los vectores y la variable de reducción */
    int A[16]= {0 , 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15};
    int B[16]= {15,14,13,12,11,10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
    int C[16]= {};
    int total=0;

    /* Se paraleliza el bucle con:
     * - Reducción de la variable total
     * - Bloques estáticos de 3 elementos
     */
    #pragma omp parallel for reduction(+:total) schedule(static,3)
    for(int i=0;i<16;i++){
        C[i] = A[i] + B[i];
        printf("C[%d]=%d\n", i, C[i]);
        total += C[i];

        int f = omp_get_thread_num();
        printf("i=%d processed by thread=%d\n", i, f);
    }

    printf("total=%d\n", total);
}
```

Los resultados obtenidos tras 100 ejecuciones son:

Serie-000 promedios:

- Tiempo real: 0.00488 segundos
- Tiempo usuario: 0.00089 segundos
- Tiempo sistema: 0.00131 segundos

Parallel-000 promedios:

- Tiempo real: 0.02742 segundos
- Tiempo usuario: 0.03137 segundos
- Tiempo sistema: 0.00366 segundos

Archivos: serie-001.c y parallel-001.c

Las instrucciones son las siguientes:

1. Calcular C como la resta de los vectores A y B más un offset
2. Distribuir las iteraciones del bucle en bloques de 5 elementos de manera estática
3. Imprimir en pantalla el número de hilo con cada resultado

El código realizado es:

```
/* parallel-001.c - Juan Sebastian Manrique Moreno
 * Cálculo de C como resta de vectores A y B más offset
 * Distribución de iteraciones en bloques de 5 elementos estáticos
 * Impresión del número de hilo por cada resultado
 */

#include <stdio.h>
#include <omp.h>

int main(){
    /* Se obtiene el número de hilo maestro y se imprime */
    int m = omp_get_thread_num();
    printf("master thread=%d in serial region\n", m);

    /* Se declaran los vectores y variables necesarias */
    int A[16]= {0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15};
    int B[16]= {15,14,13,12,11,10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
    int C[16]= {};
    int offset=33;
    int thisOffset=0;

    /* Se paraleliza el bucle con:
     * - Bloques estáticos de 5 elementos
     * - Copia privada inicial de offset
     * - Último valor de thisOffset preservado
     */
    #pragma omp parallel for schedule(static,5) firstprivate(offset) lastprivate(thisOffset)
    for(int i=0;i<16;i++){
        thisOffset = offset * i;
        C[i] = A[i] - B[i] + thisOffset;
        printf("C[%d]=%d, thisOffset=%d\n", i, C[i], thisOffset);

        int f = omp_get_thread_num();
        printf("i=%d processed by thread=%d\n", i, f);
    }

    printf("thisOffset=%d\n", thisOffset);
}
```

Los resultados obtenidos tras 100 ejecuciones son:

Serie-001 promedios:

- Tiempo real: 0.00565 segundos
- Tiempo usuario: 0.00101 segundos
- Tiempo sistema: 0.00128 segundos

Parallel-001 promedios:

- Tiempo real: 0.02856 segundos

- Tiempo usuario: 0.03422 segundos
- Tiempo sistema: 0.00373 segundos

Archivos: serie-002.c y parallel-002.c

Las instrucciones son las siguientes:

1. Calcular C como resta de matrices A y B más offset
2. Distribuir iteraciones en bloques de 5 elementos estáticamente
3. Usar cláusula que paraleliza ambos bucles como uno solo

El código realizado es:

```

/* parallel-002.c - Juan Sebastian Manrique Moreno
 * Cálculo de C como resta de matrices A y B más offset
 * Distribución de iteraciones en bloques de 5 elementos estáticos
 * Paralelización de bucles anidados como uno solo
 */

#include <stdio.h>
#include <omp.h>

int main(){
    /* Se declaran las matrices y variables necesarias */
    int A[4][4]= { { 0 , 1, 2, 3},{ 4, 5, 6, 7},{8, 9,10,11},{12,13,14,15} };
    int B[4][4]= { {15,14,13,12},{11,10, 9, 8},{7, 6, 5, 4},{ 3, 2, 1, 0} };
    int C[4][4]= { {},{},{},{} };
    int offset=33;
    int thisOffset=0;

    /* Se paralelizan ambos bucles como uno solo usando:
     * - Collapse(2) para fusionar los bucles
     * - Bloques estáticos de 5 elementos
     * - Copia privada inicial de offset
     * - Último valor de thisOffset preservado
     */
    #pragma omp parallel for collapse(2) schedule(static,5) firstprivate(offset) lastprivate(thisOffset)
    for(int i=0;i<4;i++){
        for(int j=0;j<4;j++){
            thisOffset = offset * (i*4 + j);
            C[i][j] = A[i][j] - B[i][j] + thisOffset;
            printf("C[%d][%d]=%d, thisOffset=%d\n", i, j, C[i][j], thisOffset);

            int f = omp_get_thread_num();
            printf("i=%d,j=%d processed by thread=%d\n", i, j, f);
        }
    }

    printf("thisOffset=%d\n", thisOffset);
}

```

Los resultados obtenidos tras 100 ejecuciones son:

Serie-002 promedios:

- Tiempo real: 0.006 segundos
- Tiempo usuario: 0.00109 segundos
- Tiempo sistema: 0.0013 segundos

Parallel-002 promedios:

- Tiempo real: 0.02754 segundos
- Tiempo usuario: 0.03381 segundos

- Tiempo sistema: 0.00359 segundos